

Rapport de Projet: Modélisation et Simulation d'un Robot Manipulateur à 4 Degrés de Liberté

PERDRIX Edouard - EL-GHOCHE Badria

7 octobre 2025

1 Introduction

Ce rapport évoque la modélisation et la simulation d'un robot manipulateur porte-outil à quatre degrés de liberté (4 DoF). L'objectif est de mettre en place un simulateur sous Matlab, capable d'analyser les relations géométriques directes et inverses du robot. Nous validerons ces modèles et nous aborderons le suivi de trajectoire.

Le travail s'organise autour de plusieurs étapes successives :

- Modélisation géométrique directe (MGD) : établir les équations reliant les coordonnées articulaires du robot à la position de l'outil et tracer l'espace de travail atteignable.
- Modélisation géométrique inverse (MGI) : calculer les coordonnées articulaires à partir d'une position cartésienne donnée et comparer les résultats obtenus avec ceux de la MGD.
- Validation croisée : vérifier la cohérence des deux modèles en montrant que $MGD(MGI)$ et $MGI(MGD)$ se rapprochent de l'identité.
- Génération d'une trajectoire fermée : programmer une trajectoire cartésienne (cercle, rectangle ou ellipse) à l'intérieur de l'espace de travail du robot.
- Contrôle en position articulaire : implémenter une stratégie de commande permettant de suivre la trajectoire planifiée.

Ce rapport présente la démarche suivie, les modèles théoriques utilisés ainsi que les résultats obtenus, tout en mettant en évidence les points forts et les limites de chaque approche.

2 Étapes du projet

2.1 Modélisation du robot manipulateur

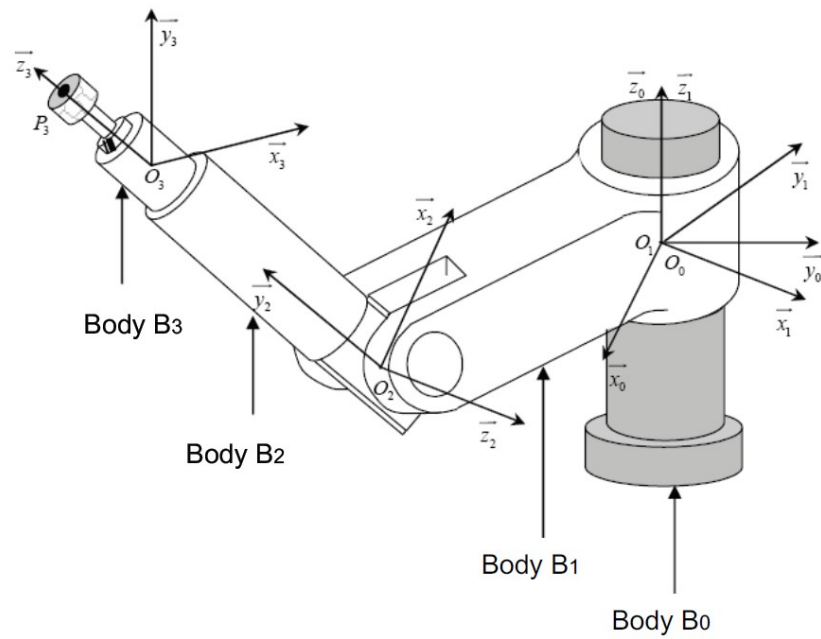
Le robot étudié est un bras robotique à quatre degrés de liberté. Pour le modéliser, nous avons utilisé la bibliothèque Robotic Tools de Matlab et les fonctions incluses dedans.

Les degrés de liberté sont les suivants :

- q_1 : rotation autour de l'axe vertical Z (articulation révolutive).
- q_2 : rotation autour de l'axe X (articulation révolutive).
- q_3 : rotation autour de l'axe Z (articulation révolutive).
- q_4 : translation selon l'axe Z (articulation prismatique).

Les paramètres géométriques principaux du robot sont :

- $a = 1.85$ m : longueur principale du bras
- $b = 0.35$ m : décalage associé à l'outil.



Design 2D du Robot

```

%% === Construction du robot graphique ===
robot = rigidBodyTree;

% q1 : rotation axe Z
B1 = rigidBody('Body1');
J1 = rigidBodyJoint('Joint1','revolute');
J1.JointAxis = [0 0 1];
setFixedTransform(J1, trvec2tform([0 0 0.2]));
B1.Joint = J1;
addBody(robot, B1, 'base');

% q2 : rotation axe X
B2 = rigidBody('Body2');
J2 = rigidBodyJoint('Joint2','revolute');
J2.JointAxis = [1 0 0];
setFixedTransform(J2, trvec2tform([0 -a 0]));
B2.Joint = J2;
addBody(robot, B2, 'Body1');

% q3 : rotation axe Z
B3 = rigidBody('Body3');
J3 = rigidBodyJoint('Joint3','revolute');
J3.JointAxis = [0 0 1];
setFixedTransform(J3, trvec2tform([0 0 b]));
B3.Joint = J3;
addBody(robot, B3, 'Body2');

% q4 : translation axe Z (prismatique)
B4 = rigidBody('Body4');
J4 = rigidBodyJoint('Joint4','prismatic');
J4.JointAxis = [0 0 1];
setFixedTransform(J4, trvec2tform([0 0 0]));
B4.Joint = J4;
addBody(robot, B4, 'Body3');

```

Construction du robot sous Matlab (RigidBodyTree)

2.2 Modélisation Géométrique Directe (MGD)

La modélisation géométrique directe (MGD) permet de déterminer la position de l'outil du robot à partir des valeurs de ses articulations. Si l'on connaît les angles et la translation des différentes articulations, on peut calculer directement la position de l'extrémité du bras dans l'espace 3D.

2.2.1 Principe théorique

Pour notre robot à 4 degrés de liberté, les articulations se composent de trois rotations (q_1 , q_2 , q_3) et d'une translation (q_4). Avec les paramètres géométriques du bras ($a=1.85$ m) et le décalage lié à l'outil ($b=0.35$ m), on obtient les équations suivantes :

$$\begin{cases} x = (a + (q_4 + b) \sin(q_2)) \sin(q_1) \\ y = (a + (q_4 + b) \sin(q_2)) \cos(q_1) \\ z = (q_4 + b) \cos(q_2) \end{cases}$$

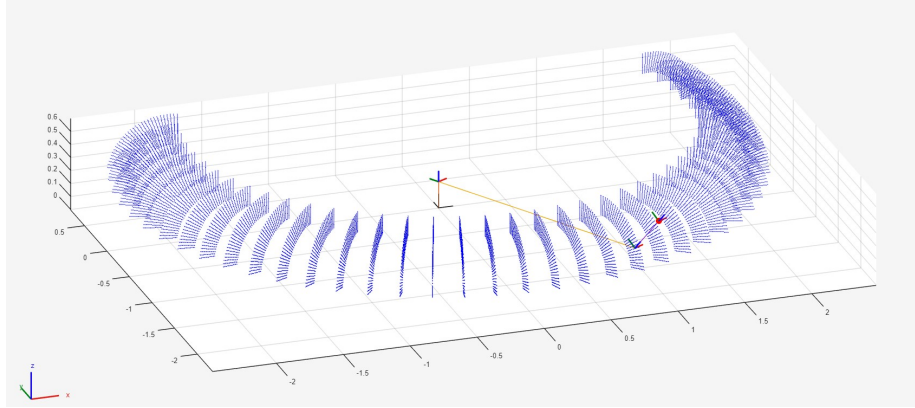
2.2.2 Implémentation dans Matlab

Pour traduire ces équations en pratique, nous avons implémenté la MGD dans Matlab à l'aide d'une fonction compacte que vous retrouvez sous le nom de $MGD(q, a, b)$ dans notre fichier Matlab.

Nous avons ensuite réalisée une interface graphique afin de rendre la simulation plus intuitive et visuelle. Elle propose quatre curseurs correspondant aux articulations q_1 , q_2 , q_3 et q_4 . Lorsque l'on déplace un curseur, la configuration du robot se met à jour instantanément et l'organe terminal (représenté par un point rouge) se déplace en conséquence dans l'espace 3D. Ce dispositif est particulièrement intéressant car il permet d'expérimenter directement l'effet de chaque articulation : on peut voir, en temps réel, comment un changement d'angle ou de translation modifie la position du robot. Vous pouvez retrouver notre code dans l'espace interface utilisateur dans notre fichier Matlab

2.2.3 Résultats et observations

Le nuage de points généré représente toutes les positions atteignables par l'outil du robot, en respectant les limites de ses articulations. L'interface interactive confirme la validité du modèle, chaque mouvement des curseurs est immédiatement traduit par un déplacement cohérent de l'organe terminal. Cette étape a permis non seulement de valider le modèle géométrique direct, mais aussi de disposer d'un outil pédagogique efficace pour visualiser la cinématique du robot.



Espace de travail du robot

2.3 Modélisation Géométrique Inverse (MGI)

La modélisation géométrique inverse sert à déterminer les valeurs des coordonnées articulaires $\{q_1, q_2, q_3, q_4\}$ à partir d'une position cartésienne cible $\{x, y, z\}$ de l'outil du robot. L'objectif est donc de trouver les angles de rotation

et la translation nécessaires pour que l'effecteur atteigne une position donnée dans l'espace de travail.

2.3.1 Principe théorique

On obtient les équations suivantes :

$$\begin{cases} q1 = \arctan^2(y, x) \\ q2 = \arctan^2((\sqrt{x^2 + y^2}) - a, z - 0.2) \\ q3 = 0 \\ q4 = \sqrt{(((\sqrt{x^2 + y^2}) - a)^2 + (z - 0.2)^2) - b} \end{cases}$$

2.3.2 Implémentation dans Matlab

Nous avons programmé l'algorithme de calcul du MGI, puis implémenté dans l'environnement Matlab. Ce programme permet, à partir des équations géométriques établies, de calculer les variables articulaires correspondantes à une position cartésienne. Vous pouvez retrouver notre fonction sous le nom MGI(p3, a, b) dans notre fichier Matlab.

```
% --- Tâche 2 : mise à jour par curseurs cartésiens ---
function updateFromXYZ(~,~)
    x = sldX.Value;
    y = sldY.Value;
    z = sldZ.Value;
    q = MGI([x, y, z], a, b);
    config = homeConfiguration(robot);
    config(1).JointPosition = q(1);
    config(2).JointPosition = q(2);
    config(3).JointPosition = q(3);
    config(4).JointPosition = q(4);
    show(robot, config, 'Parent', ax, 'PreservePlot', false, 'Frames', 'on');
    set(hEff, 'XData', x, 'YData', y, 'ZData', z);
    % synchronisation curseurs articulaires
    sld1.Value = min(max(rad2deg(q(1)), sld1.Limits(1)), sld1.Limits(2));
    sld2.Value = min(max(rad2deg(q(2)), sld2.Limits(1)), sld2.Limits(2));
    sld3.Value = min(max(rad2deg(q(3)), sld3.Limits(1)), sld3.Limits(2));
    sld4.Value = min(max(100*q(4), sld4.Limits(1)), sld4.Limits(2));
end
```

Fonction pour effectuer la tâche 2

2.4 Validation croisée MGD MGI

La validation croisée consiste à vérifier que les modèles géométriques direct (MGD) et inverse (MGI) sont cohérents entre eux.

2.4.1 Principe théorique

La validation croisée consiste à vérifier que les modèles géométriques direct (MGD) et inverse (MGI) sont cohérents entre eux. Pour cela, nous appliquons successivement les deux modèles :

- MGD(MGI) : À partir d'une position cartésienne, nous calculons les coordonnées articulaires avec le MGI, puis nous utilisons ces coordonnées pour retrouver la position cartésienne avec le MGD.
- MGI(MGD) : À partir de coordonnées articulaires, nous calculons la position cartésienne avec le MGD, puis nous utilisons cette position pour retrouver les coordonnées articulaires avec le MGI.

L'objectif est de montrer que ces transformations successives se rapprochent de l'identité, c'est-à-dire que les erreurs entre les valeurs initiales et finales sont minimales.

2.4.2 Implémentation dans Matlab

La fonction implémentée calcule les erreurs entre les coordonnées articulaires et cartésiennes avant et après les transformations successives.

```
% --- Tâche 3 : validation croisée ---
function validateCross(~,~)
    q1 = deg2rad(30);
    q2 = deg2rad(40);
    q3 = 0;
    q4 = 0.12;
    p3 = MGD([q1, q2, q3, q4], a, b);
    qd = MGI(p3, a, b);
    p3_check = MGD(qd, a, b);
    errQ = [rad2deg(q1-qd(1)), rad2deg(q2-qd(2)), q3-qd(3), (q4-qd(4))*100];
    errXYZ = [p3(1)-p3_check(1), p3(2)-p3_check(2), p3(3)-p3_check(3)];
    uialet(' ', sprintf('=== Validation ===\nErreur q : [%2f°, %2f°, %2f°, %2f rad, %2f cm]\nErreur pos : [%2e, %2e, %2e] m', ...
        errQ(1), errQ(2), errQ(3), errQ(4), errXYZ(1), errXYZ(2), errXYZ(3)), ...
        'Validation MGD <-> MGI');
end
```

Fonction pour effectuer la tâche 3

2.4.3 Résultats et observations

Les résultats montrent que les erreurs entre les valeurs initiales et finales sont très faibles, ce qui confirme la cohérence des modèles MGD et MGI. Par exemple, pour une configuration initiale donnée, les erreurs sur les coordonnées articulaires sont de l'ordre de quelques degrés ou centimètres, et les erreurs sur les coordonnées cartésiennes sont de l'ordre de 10^{-15} mètres. Cela valide la précision des modèles et permet de passer à l'étape suivante : le suivi de trajectoire.

2.5 Génération d'une trajectoire fermée

L'objectif de cette étape est de programmer une trajectoire cartésienne fermée (cercle, rectangle ou ellipse) à l'intérieur de l'espace de travail du robot. Cette trajectoire doit être atteignable par le robot, c'est-à-dire que chaque point de la trajectoire doit se situer dans l'espace de travail calculé précédemment.

2.5.1 Principe théorique

Pour notre projet, nous avons choisi de programmer une trajectoire circulaire. Les coordonnées cartésiennes du cercle sont définies par :

- Un centre (x_c, y_c, z_c) situé dans l'espace de travail.
- Un rayon R adapté à la taille de l'espace de travail.

2.5.2 Implémentation dans Matlab

Nous avons implémenté une fonction pour tracer un cercle et faire suivre cette trajectoire au robot. Voici le code utilisé :

```
% --- Tâche 4 : tracer un cercle ---
function trajCercle(~, ~)
% Centre et rayon du cercle (ajusté sur le nuage bleu)
xc = -1.70;
yc = -1.10;
zc = 0.55; % Assurez-vous que cette valeur est dans la plage atteignable par le robot
R = 0.20;
theta = linspace(0, 2*pi, 60);
X = xc + R*cos(theta);
Y = yc + R*sin(theta);
Z = zc*ones(size(theta));
plot3(ax, X, Y, Z, 'g--', 'LineWidth', 1.5);
for i = 1:length(X)
    q = MGI([X(i), Y(i), Z(i)], a, b);
    config = homeConfiguration(robot);
    config(1).JointPosition = q(1);
    config(2).JointPosition = q(2);
    config(3).JointPosition = q(3);
    config(4).JointPosition = q(4);
    show(robot, config, 'Parent', ax, 'PreservePlot', false, 'Frames', 'on');
    set(hEff, 'XData', X(i), 'YData', Y(i), 'ZData', Z(i));
    plot3(ax, X(i), Y(i), Z(i), 'r.', 'MarkerSize', 8);
    pause(0.05);
end
end
```

Fonction pour effectuer la tâche 4

2.5.3 Résultats et observations

La trajectoire circulaire est correctement tracée dans l'espace de travail du robot. Le robot suit chaque point de la trajectoire de manière fluide et précise, ce qui montre que le modèle géométrique inverse est capable de calculer les coordonnées articulaires nécessaires pour atteindre chaque point cartésien de la trajectoire. Cela valide également la cohérence entre les modèles MGD et MGI.

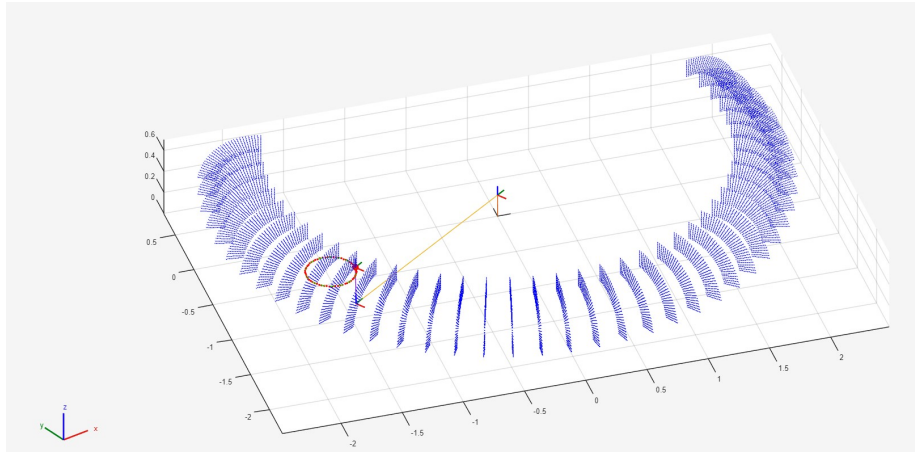
2.6 Contrôle en position articulaire

Cette étape consiste à implémenter une stratégie de commande permettant de suivre la trajectoire planifiée. L'idée est de contrôler les articulations du robot de manière à ce qu'il suive la trajectoire circulaire définie précédemment.

Pour cela, nous avons utilisé les curseurs articulaires pour simuler un contrôle en position. Chaque point de la trajectoire est converti en coordonnées articulaires grâce au MGI, puis ces coordonnées sont appliquées au robot pour qu'il atteigne chaque point de la trajectoire.

2.6.1 Implémentation dans Matlab

Nous avons utilisé les fonctions de rappel des curseurs pour mettre à jour la position du robot en temps réel.



Suivi de trajectoire

2.7 Visualisation des signaux articulaires

Cette étape consiste à visualiser les signaux articulaires, c'est-à-dire les variations des angles et de la translation des articulations en fonction du temps ou des itérations. Cela permet d'analyser le comportement dynamique du robot lors du suivi de trajectoire.

2.7.1 Implémentation dans Matlab

Nous avons implémenté une fonction pour tracer les signaux articulaires lors du suivi de la trajectoire circulaire.

```

% --- Tâche 5 : visualisation des signaux articulaires ---
function trajSignaux(~, ~)
% Paramètres du cercle
xc = -1.70; % Centre en X
yc = -1.10; % Centre en Y
zc = 0.55; % Centre en Z (ajusté pour être dans l'espace de travail)
R = 0.20; % Rayon du cercle
theta = linspace(0, 2*pi, 100); % Discrétisation du cercle

% Coordonnées cartésiennes du cercle
X = xc + R*cos(theta);
Y = yc + R*sin(theta);
Z = zc*ones(size(theta));

% Initialisation des vecteurs pour les signaux articulaires
Q1 = []; % q1 en radians
Q2 = []; % q2 en radians
Q3 = []; % q3 en radians
Q4 = []; % q4 en mètres

% Calcul des coordonnées articulaires pour chaque point du cercle
for i = 1:length(X)
    q = MGL([X(i), Y(i), Z(i)], a, b);
    Q1(end+1) = q(1);
    Q2(end+1) = q(2);
    Q3(end+1) = q(3);
    Q4(end+1) = q(4);
end

% Temps ou itérations (pour l'axe des abscisses)
temps = linspace(0, 2*pi, length(X));

% Affichage des signaux articulaires en fonction du temps
figure;
subplot(4,1,1);
plot(temps, rad2deg(Q1), 'linewidth', 1.5, 'Color', 'b');
ylabel('q1 (°)');
title('Evolution des angles articulaires');
grid on;

subplot(4,1,2);
plot(temps, rad2deg(Q2), 'linewidth', 1.5, 'Color', 'r');
ylabel('q2 (°)');
grid on;

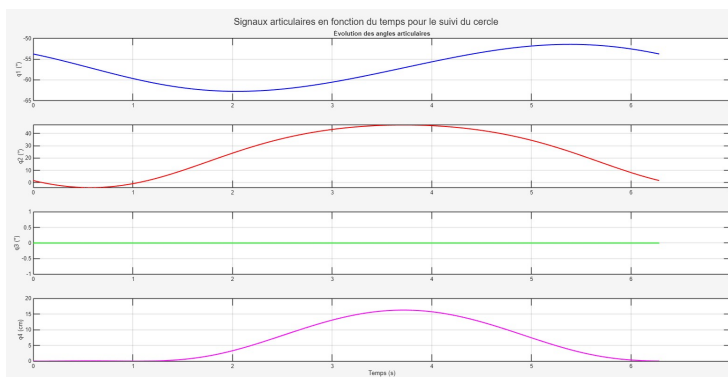
subplot(4,1,3);
plot(temps, rad2deg(Q3), 'linewidth', 1.5, 'Color', 'g');
ylabel('q3 (°)');
grid on;

subplot(4,1,4);
plot(temps, Q4*100, 'linewidth', 1.5, 'Color', 'm');
ylabel('q4 (cm)');
xlabel('Temps (s)');
grid on;

sgtitle('Signaux articulaires en fonction du temps pour le suivi du cercle');
end

```

Fonction pour effectuer la tâche 5



Graphique des signaux articulaires

2.7.2 Résultats et observations

Les graphiques montrent les variations des angles q_1 , q_2 , q_3 et de la translation q_4 lors du suivi de la trajectoire circulaire. Ces signaux permettent de visualiser comment chaque articulation évolue pour suivre la trajectoire. Par exemple, q_1 et q_2 varient de manière sinusoïdale, ce qui est cohérent avec le mouvement circulaire. q_3 reste constant à 0, comme prévu, et q_4 varie légèrement pour ajuster la hauteur de l'outil.

3 Conclusion

Ce projet a permis de modéliser et simuler un robot manipulateur à quatre degrés de liberté sous Matlab. Nous avons établi les modèles géométriques direct et inverse, validé leur cohérence, et implémenté une stratégie de suivi de trajectoire.

3.1 Points forts

- La modélisation géométrique directe et inverse a été validée avec succès, montrant une excellente cohérence entre les deux modèles.
- L'interface graphique interactive permet une visualisation claire et intuitive du comportement du robot.
- Le suivi de trajectoire circulaire a été implémenté avec succès, démontrant la capacité du robot à atteindre des positions précises dans son espace de travail.

3.2 Limites et perspectives

- Les modèles actuels supposent des mouvements idéaux sans tenir compte des contraintes dynamiques ou des limites physiques des actionneurs.
- Une extension possible serait d'intégrer un contrôleur PID pour améliorer la précision du suivi de trajectoire.
- Une autre perspective serait d'ajouter des obstacles dans l'espace de travail et de planifier des trajectoires pour les éviter.

En conclusion, ce projet a permis de développer une compréhension approfondie de la cinématique des robots manipulateurs et de mettre en pratique des concepts théoriques à travers une simulation interactive.